

Práctico 3: Arquitectura, Escalabilidad y Gestores NoSQL

ACTIVIDAD 1: Solución de Infraestructura (Caso: "Ticket-All")

1. La propuesta de la empresa de comprar un servidor más potente es una estrategia de Escalabilidad Vertical. Esta solución tiene un límite físico y económico, ya que no se pueden añadir recursos de hardware (CPU/RAM) de forma indefinida. Además, persiste el riesgo de un único punto de fallo: si ese servidor colapsa, todo el sistema cae. Para un escenario de alta concurrencia como la venta masiva de entradas, es preferible una Escalabilidad Horizontal, que consiste en añadir servidores estándar a un clúster para distribuir la carga, permitiendo un crecimiento casi ilimitado y mayor adaptabilidad.
2. Para distribuir equitativamente a los miles de compradores, se propone una estrategia de Particionamiento por Hash utilizando el `id_usuario` como Shard Key. El uso de un hash asegura una distribución uniforme y aleatoria de los usuarios en todos los nodos del clúster. No convendría usar el `id_evento`, ya que todos los compradores se dirigirían al mismo shard al mismo tiempo, generando un "Punto Caliente" (Hotspot).

¿Qué es un Hotspot?: Es una saturación en un solo servidor debido a una clave de fragmentación mal elegida (como una fecha o ID secuencial), donde un nodo recibe toda la carga de escritura mientras los demás están ociosos

3. Los replica sets permiten que, si el nodo primario falla, un nodo secundario tome su lugar. Esto asegura que el sistema siga funcionando sin interrupciones importantes durante la venta.

Nodo Primario: Es el único que recibe las operaciones de escritura y actúa como fuente principal de datos.

Nodos Secundarios: Replican los datos del primario y pueden atender operaciones de lectura para balancear la carga. Si el servidor primario falla durante la venta, los secundarios inician un proceso automático de "elección" para designar un nuevo primario, permitiendo que el sistema siga operativo sin interrupciones críticas para los usuarios.

ACTIVIDAD 2: Modelado y Consistencia (Caso: "Fintech-Plus")

1. Para saldos y transferencias se debe aplicar el modelo ACID con Consistencia Inmediata. Al ser datos críticos, es indispensable garantizar que cualquier lectura devuelva siempre el último dato escrito, evitando errores en los saldos bancarios.

Para el muro de noticias): Se aplica el modelo BASE con Consistencia Eventual. En este caso, se prioriza la disponibilidad del sistema sobre la actualización instantánea; los comentarios o "likes" se volverán consistentes en todas las réplicas después de un breve periodo de tiempo.

2. En lugar de usar JOINS, se puede desnormalizar guardando los datos del usuario dentro del comentario. Esto mejora la velocidad de lectura, aunque implique duplicación de información.
3. El flujo de consulta sería: cliente -> backend -> mongos -> config servers -> shard -> respuesta. Mongos actúa como router y los config servers indican dónde están los datos.

ACTIVIDAD 3: Investigación

1. El Teorema CAP es el marco que nos permite clasificar y elegir la base de datos ideal para un proyecto basándose en tres atributos de calidad que entran en conflicto en sistemas distribuidos:
 - Consistencia (C): Garantiza que todos los clientes vean los mismos datos al mismo tiempo, sin importar a qué nodo se conecten.
 - Disponibilidad (A): Asegura que cualquier solicitud reciba una respuesta válida en un tiempo razonable, incluso si algunos nodos están inactivos.
 - Tolerancia a Particiones (P): Capacidad del sistema de seguir operando a pesar de interrupciones o retrasos en la comunicación entre sus nodos.

¿Por qué no se pueden tener las tres al mismo tiempo?

El teorema sostiene que, ante una Partición de red (P) —que es inevitable en sistemas distribuidos—, el diseñador debe elegir obligatoriamente entre Consistencia o Disponibilidad. Si se elige Consistencia (CP), el sistema debe bloquearse o dar error si no puede asegurar que todos los nodos tienen el último dato; si se elige Disponibilidad (AP), el sistema responderá rápido pero podría entregar datos obsoletos mientras se resincroniza.

Relación con BASE: Mientras que las bases SQL tradicionales buscan el modelo ACID (priorizando Consistencia), el principio BASE es la respuesta práctica para los sistemas que eligen la combinación AP (Disponibilidad y Tolerancia a Particiones). BASE acepta que el sistema esté en un "estado suave" y sea "eventualmente consistente", priorizando que el usuario siempre reciba una respuesta (Disponibilidad) por encima de la exactitud inmediata de los datos.

2. Redis es considerablemente más rápido que MongoDB para gestionar tablas de posiciones (leaderboards) en tiempo real debido a dos pilares fundamentales: su arquitectura in-memory y sus estructuras de datos especializadas como los Sorted Sets.

A continuación, se detalla esta comparativa basada en fuentes técnicas:

¿Por qué Redis supera a MongoDB en Leaderboards?

Almacenamiento en RAM vs. Disco: Redis almacena todos sus datos directamente en la memoria RAM, lo que permite operaciones de lectura y escritura con latencias inferiores a un milisegundo (sub-milisegundo). En contraste, aunque MongoDB cachea datos en RAM, su motor principal está basado en disco, lo que introduce una latencia mayor (típicamente de 1-10ms) debido a la gestión de entrada/salida de archivos.

Estructuras de Datos Nativas (Sorted Sets): Esta es la ventaja decisiva. Redis posee un tipo de dato llamado Sorted Set (ZSET) que mantiene automáticamente los elementos ordenados por una puntuación (score).

En MongoDB, para obtener un ranking, el motor debe consultar documentos, aplicar índices y realizar un ordenamiento manual o mediante agregaciones, lo que consume tiempo de CPU.

Pichulman Miguel Angel

En Redis, el ordenamiento es "gratis" y constante; cada vez que se añade o actualiza un puntaje, la posición se recalcula internamente con una complejidad logarítmica $O(\log N)$, permitiendo consultas de ranking instantáneas incluso con millones de usuarios.

Comandos de Redis para Leaderboards

Para gestionar una tabla de posiciones, no se suelen usar comandos básicos como LPUSH o SADD (que no manejan orden), sino comandos específicos de Sorted Sets. Aquí se explican tres esenciales:

ZADD: Agrega uno o más miembros a la tabla de posiciones con sus respectivos puntajes. Si el usuario ya existe, actualiza su puntaje y su posición en el ranking de forma automática.

Ejemplo: ZADD ranking 1500 "usuario_1" (asigna 1500 puntos a "usuario_1").

ZINCRBY: Incrementa el puntaje de un miembro específico de forma atómica. Es ideal para actualizar la puntuación en tiempo real (por ejemplo, tras ganar una partida) sin necesidad de leer el valor previo.

Ejemplo: ZINCRBY ranking 50 "usuario_1" (suma 50 puntos al total actual del usuario).

ZRANGE (con opción REV): Devuelve un rango de miembros ordenados por su puntaje. Al usar la opción REV, se obtienen los puntajes de mayor a menor, facilitando la creación del "Top 10" de la competencia.

Ejemplo: ZRANGE ranking 0 9 REV WITHSCORES (obtiene los 10 mejores jugadores con sus puntos).

3. Neo4j y la estructura de Grafos: Empresas especializadas prefieren Neo4j para la detección de fraude debido a su modelo de nodos y aristas, diseñado específicamente para almacenar y navegar relaciones entre entidades. A diferencia de los documentos de MongoDB, Neo4j permite realizar consultas complejas sobre conexiones (como redes de contactos o transferencias sospechosas circulares) de manera mucho más eficiente y natural mediante su lenguaje Cypher.

Pichulman Miguel Angel

Bibliografía:

OpenWebinars. (s.f.). *¿Qué es el Teorema CAP y cómo afecta al elegir la base de datos?* Recuperado de: <https://openwebinars.net/blog/que-es-el-teorema-cap-y-como-afecta-al-elegir-la-base-de-datos/>

Redis Software. (s.f.). Leaderboards with Redis Sorted Sets. Redis Documentation. Recuperado de: <https://redis.io/docs/latest/develop/data-types/probabilistic/leaderboards/>

Dev Community. (2024). In-memory vs Disk-based databases: Performance Comparison for Real-time Applications. Recuperado de: <https://dev.to/>

Neo4j Graph Data Platform. (s.f.). Fraud Detection with Graph Databases. Recuperado de: <https://neo4j.com/use-cases/fraud-detection/>